

CARKIT 88C0 GMSL User Guide

1	Introduction	2
1.1	Limitations of this release.....	2
2	Content of package	3
3	Pre-requisites.....	4
3.1	Hardware	4
3.2	Software.....	4
4	Hardware Setup.....	5
5	Getting started	7
5.1	Install package dependencies	7
5.2	After reboot	7
6	Features	8
6.1	Control and configuration interface.....	8
6.2	Raw data streaming	8
6.2.1	Using v4l2-ctl application	8
6.2.2	Memory-mapped stream via a custom application	9
6.3	Control from Matlab host PC	10
7	Software environment	11
7.1	Custom video driver	11
7.2	Serializer/De-serializer configuration	11
7.3	iRFE-Jetson.....	12
7.4	Strata	12

1 Introduction

The scope of this User Guide is to describe the usage of CARKIT 88C0 together with the Nvidia Jetson AGX Orin and MAX96717-MAX9296A serializer-deserializer.

Please see the chapter Pre-requisites for further information about target system and used software versions.

This user guide describes the current stable release. The system still has limitations, both in hardware and software, and is not intended for productive use and should only be used for evaluation purposes.

Infineon will provide continuous software updates.

1.1 Limitations of this release

Please read and acknowledge these limitations carefully. Limitations are planned to be removed in upcoming releases.

- Radar application cycle is controlled from user space
 - Timing of application cycle by use of user space "sleep" API
 - Results in reduced accuracy
 - The timing will be improved in future versions
- First frame after transition from Low Power to Operation state is not correctly transmitted when using non-continuous clock mode
 - Default examples all use continuous clock mode
- First frame after CTRX reset is not correctly transmitted.
 - Default examples do not read the first frames.

2 Content of package

This is the content of the Infineon-provided release package.

- *install_jetson_dependencies.sh*
 - Install all necessary runtime dependencies
- *init_jetson.py*
 - Initialize runtime components i.e. load drivers (video, GMSL, SPI)
- **artifacts**
 - *tegra234-p3701-0000-p3737-0000-carkit88c0-gmsl.dtb*
 - Device tree with changes needed to support GMSL and correctly map CSI-2
 - *pinmux_config_gmsl_max9296.sh*
 - Script to establish the required pinmux settings for GMSL on Jetson
 - *spi-tegra114.ko*
 - SPI driver with performance fixes
- **scripts**
 - RSPS
 - Script to visualize raw data (with FFT calculation) recorded with v4l2-ctl (see below)
 - *capture_video0_4096x512.sh*
 - Script to capture the CSI-2 data stream on /dev/video0 with the format 4096x512 (see below)
 - *tracing.sh*
 - Enables and prints the kernel trace
- **src**
 - *demo_app*
 - iRFE driver with demo application
 - modules
 - max929x
 - source code of kernel modules for GMSL ser/des
 - radar
 - source code for the custom video driver to receive radar data via CSI-2
- **strata-board-simulation**
 - Strata Jetson-side implementation
 - This is needed to interact with the system using Matlab/Python from an Ethernet-connected PC
- **documentation**
 - The software user guide documentation

3 Pre-requisites

3.1 Hardware

This package requires the following individual hardware components

- **Nvidia Jetson AGX Orin development kit**
- **ADAS_CARKIT_2C0_G_DES deserializer board**
 - Based on Analog® MAX9296A.
- **CARKIT 88C0 module(s)**
 - Including Coax cable
 - Up to two radar modules can be connected simultaneously

 Please get in contact with your local Infineon sales representative on details how to obtain the individual hardware components

3.2 Software

All software has been developed for and targets Nvidia Jetson release 35.4.1. Please make sure you run Jetpack 5.1.2 on the Jetson before proceeding with setup and operation.

To check your current version please run the following command on your Jetson

```
cat /etc/nv_tegra_release

# Should yield: # R35 (release), REVISION: 4.1, GCID: 33958178, BOARD: t186ref, EABI:
aarch64, DATE: Tue Aug 1 19:57:35 UTC 2023
```

If not running Jetpack 5.1.2, please refer to Nvidia documentation (SDK Manager) on how to reflash the Jetson:

- <https://developer.nvidia.com/embedded/learn/get-started-jetson-agx-orin-devkit>
- <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>
- <https://developer.nvidia.com/embedded/jetpack-sdk-512>

4 Hardware Setup

Below pictures gives an overview over the hardware architecture. Please refer to the Hardware User guide for detailed descriptions.

Two operation modes are supported:

- **Single Link Mode:** One CARKIT_88C0 module connected to Link A on the Deserializer board
- **Splitter Mode:** Two CARKIT_88C0 modules connected to Link A & B on the Deserializer board

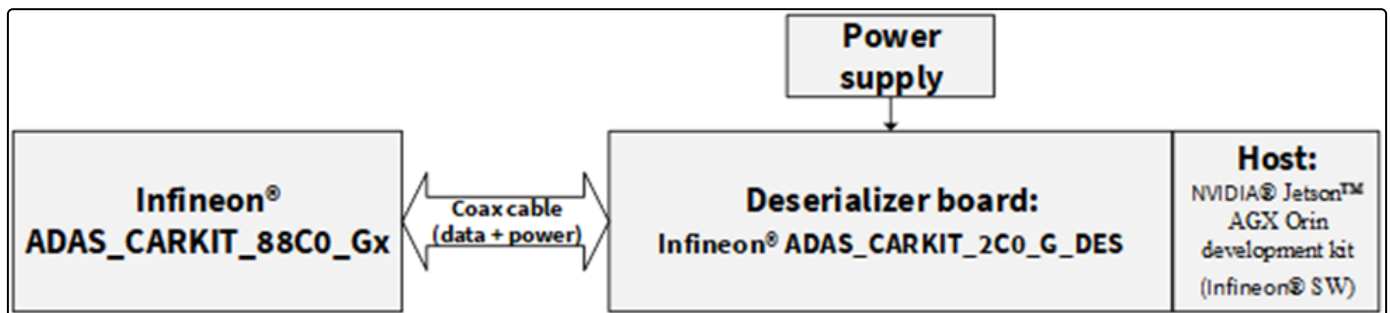
Before powering the Jetson make sure the Deserializer board is powered. The Deserializer board will provide power to the CARKIT 88C0 module over the connected coax cable (Power over GMSL). Post power up check if the CARKIT 88C0 is detectable over i2c via the i2c detect commands:

```
i2cdetect -r -y 2
```

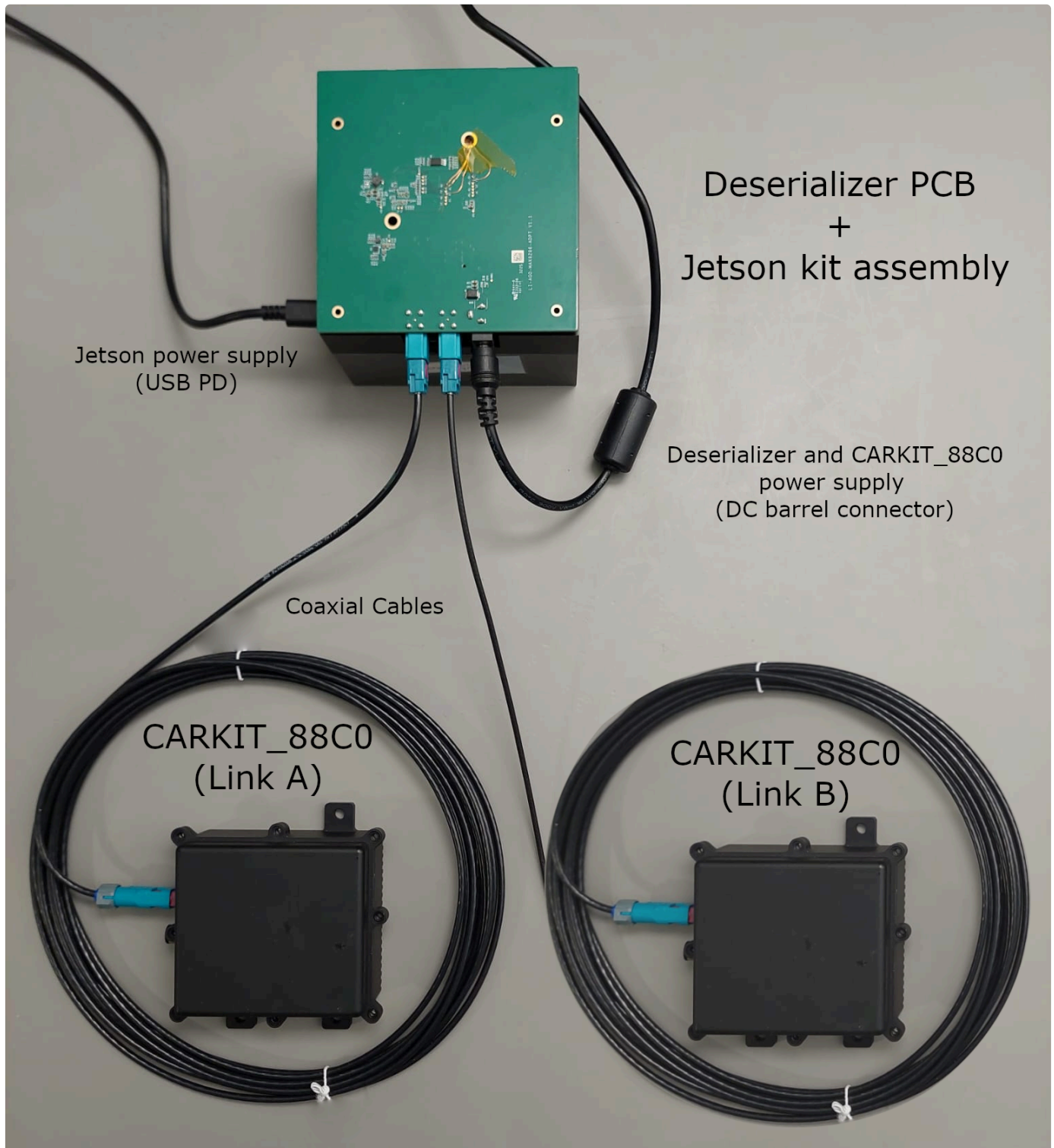
The following I2C devices should be detected:

- Deserializer at 0x48
- CARKIT_88C0 (Link A) at 0x42
- CARKIT_88C0 (Link B) at 0x43

Note: Output of i2cdetect for the CARKIT modules should read *UU*



1 Hardware architecture for single link mode



2 Hardware setup for splitter mode

5 Getting started

The following section describes the procedure how to get the software up and running.

Please make sure your Jetson meets the software prerequisites before continuing.

5.1 Install package dependencies

 The following steps needs to be performed only once after flashing the Jetson.

1. Manually install the device tree file

I. Copy the provided device tree binary (dtb) into the `/boot` directory of your Jetson (root privileges required)

```
sudo cp artifacts/tegra234-p3701-0000-p3737-0000-carkit88c0-gmsl.dtb /boot
```

II. Adapt the boot options configuration file `/boot/extlinux/extlinux.conf` to ensure the device tree is used after reboot. A valid configuration could look like this

```
TIMEOUT 30
DEFAULT carkit-88c0

MENU TITLE L4T boot options

LABEL carkit-88c0
    MENU LABEL Carkit88c0
    LINUX /boot/Image
    FDT /boot/tegra234-p3701-0000-p3737-0000-carkit88c0-gmsl.dtb
    INITRD /boot/initrd
    APPEND ${cbootargs} root=/dev/mmcblk0p1 rw rootwait rootfstype=ext4
mminit_loglevel=4 console=ttyTCU0,115200 console=ttyAMA0,115200 console=tty0
firmware_class.path=/etc/firmware fbcon=map:0 net.ifnames=0
```

2. To install the required software dependencies and tools, execute **`install_jetson_dependencies.sh`** on your Jetson

3. Reboot the Jetson

5.2 After reboot

 This step needs to be performed once after every reboot of Jetson or after disconnecting the power supply of the MAX9296 adapter board (e.g. when changing the hardware setup)

To initialize hardware and software components, execute **`init_jetson.py`** on your Jetson

This step will configure GMLS, load device drivers and establish the required pinmux settings

Jetson is now ready to communicate with the RADAR module and receive data.

6 Features

CARKIT 88C0 software supports these use cases which are described further:

- Control and configuration interface to CARKIT88C0 via iRFE
- Raw data streaming over v4l2 (Video4Linux) framework
 - File (v4l2-utils)
 - Memory-mapped stream (v4l2 API)
- Control from Matlab/Python host PC

All use cases support both modes of operation (i.e. Single Link or Splitter Mode) respectively.

i The provided CARKIT 88C0 software acts as a starting point for development of a centralized Radar sensor. Please study carefully all provided examples and the accompanying documentation e.g. CTRX8188 User Manual.

6.1 Control and configuration interface

Using the Infineon-provided iRFE driver the user can configure and control both CTRX on the CARKIT 88C0, download RAM firmware and ramp designs.

The iRFE driver enables users to write their own custom C-applications for configuration and control of the CARKIT 88C0.

The iRFE provides access to all firmware functions of the CTRX. For further information on the configuration flow of CTRX and specific firmware functions please consult the CTRX8188 User Manual.

The iRFE uses the Linux *spidev* API to communicate over SPI with CTRX. The SPI traffic is tunneled over the GMSL2 link to the CARKIT 88C0.

This release includes example applications to get you started with the iRFE.

- *carkit88c0/example*: Example showing the basic configuration flow of the standalone CTRX system and will then enter a loop to trigger X amount of frames
- *carkit88c0/example_splitter_mode*: Application example demonstrating splitter mode

The application can be compiled using the accompanying Makefile.

Please note: Application do not yet fully implement a productive Radar cycle. Infineon will provide an example in the next release.

6.2 Raw data streaming

Raw data streaming can be utilized over the Video4Linux (V4L2) API after the CARKIT 88C0 has been configured using custom iRFE code.

The provided device tree maps CTRX8188 from single link to */dev/video0*. If using dual-link operation (splitter mode) the CTRX8188 from Link B is additionally mapped to */dev/video1*.

Infineon provides two examples to interact with the V4L2 API:

6.2.1 Using v4l2-ctl application

Using the *v4l2-ctl* application the user can stream data to a binary file.

The release includes an example script to start v4l2-ctl application and an example Python script to decode the binary data to a Matlab file.

To start v4l2-ctl in verbose mode:

```
# Video device 0
v4l2-ctl -d /dev/video0 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl
bypass_mode=0 --stream-mmap --verbose

# Video device 1 (relevant for splitter mode)
v4l2-ctl -d /dev/video1 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl
bypass_mode=0 --stream-mmap --verbose
```

To start v4l2-ctl to stream to file:

```
# Video device 0
v4l2-ctl -d /dev/video0 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl
bypass_mode=0 --stream-mmap --stream-to=ctrx0_raw.bin

# Video device 1 (relevant for splitter mode)
v4l2-ctl -d /dev/video1 --set-fmt-video=width=4096,height=512,pixelformat=RG12 --set-ctrl
bypass_mode=0 --stream-mmap --stream-to=ctrx1_raw.bin
```

The resulting file will contain RAW data as outputted by Nvidia Video Input (VI) block. Notice that regardless of bit size on CTRX (RAW10, RAW12, RAW14), the Video Input will pad all values to 16 bit.

The Frame is organized as follows: | Sample1_Rx1, Sample1_Rx2, Sample1_Rx3, Sample1_Rx4, Sample1_Rx5,, Sample2_Rx1,, EntryN_Rx4 |

To load the files use the Python scripts in '*scripts*' folder. These scripts demonstrate loading the data and doing some basic processing (FFT) on them.

6.2.2 Memory-mapped stream via a custom application

This release supports streaming data over a memory-mapped stream using the V4L2 API directly from a custom application.

The configuration of the streaming interface has to match the configured ramp design in terms of samples and ramps. Failure to do so will result in no data reception.

As a starting point one can use the *V4LDataFrameCapture.cpp* class. In this class the following steps are shown

- Open and initialize /dev/videoX device
- Configure device for Memory Mapped streaming of RAW data, using the supplied video format (Video Resolution = 8*number_samples X number_ramps)
- Get (*dequeue*) a frame from V4L2 buffer
- Remove 16-bit padding (*processRawData*) to mask ADC raw samples

The class will return a Frame in the following (native CTRX) format:

```
// The Frame is organized as follows:
// | Entry1_Rx1, Entry1_Rx2, Entry1_Rx3, Entry1_Rx4, Entry2_Rx1, ....., EntryN_Rx4 |
// After retrieving one single receiver that leaves us with this layout for each of them:
// Assuming nSamples = 4, nRamps = 2:
```

```
// -----  
// |           RAMP           |           RAMP           |  
// |SAMPLE SAMPLE SAMPLE SAMPLE|SAMPLE SAMPLE SAMPLE SAMPLE|
```

Additionally in the file `CtrxTimeDataNode.cpp` it is shown how to re-format the Frame into a common (Rx) (Ramps)(Samples) multi-dimensional array (*processFrameData*)

```
receivers[receiverIndex].ramps[ramp].samples[sample]
```

6.3 Control from Matlab host PC

Using the provided board simulator, Matlab can be used to control CARKIT 88C0 and receive data directly to Matlab over Ethernet via Strata.

Note that this use case is inherently limited by the Ethernet interface and thus will have limited performance for real-time data transfers. See chapter Strata.

7 Software environment

Infineon provides the following software to operate CARKIT 88C0

- Custom video driver to receive CSI-2 data
- Kernel module to configure tunneling services over GMSL2
- iRFE-Jetson, a custom SPI and I2C driver to control CARKIT 88C0 from Jetson
- Strata, a MATLAB/Python interface to a host PC connected to Jetson through Ethernet

7.1 Custom video driver

The radar video driver (radar.ko) is provided as source code in this package. The driver will be compiled and loaded automatically by *init_jetson.py*.

The driver is implemented as minimal camera driver that enables application to receive radar data via the video4linux framework.

The current mode table is implemented and aligned with the device tree (Note: Width = 8*Number of Samples, Height=Number of Ramps)

Mode table:

```
static radar_reg *mode_table[] = {
    [RADAR_MODE_START_STREAM] = radar_start,
    [RADAR_MODE_STOP_STREAM] = radar_stop,
    [RADAR_MODE_1024x256_CROP_30FPS] = radar_1024x256_crop_30fps,
    [RADAR_MODE_2048x256_CROP_30FPS] = radar_2048x256_crop_30fps,
    [RADAR_MODE_4096x256_CROP_30FPS] = radar_4096x256_crop_30fps,
    [RADAR_MODE_8192x256_CROP_30FPS] = radar_8192x256_crop_30fps,
    [RADAR_MODE_16384x256_CROP_30FPS] = radar_16384x256_crop_30fps,
    [RADAR_MODE_1024x512_CROP_30FPS] = radar_1024x512_crop_30fps,
    [RADAR_MODE_2048x512_CROP_30FPS] = radar_2048x512_crop_30fps,
    [RADAR_MODE_4096x512_CROP_30FPS] = radar_4096x512_crop_30fps,
    [RADAR_MODE_8192x512_CROP_30FPS] = radar_8192x512_crop_30fps,
    [RADAR_MODE_16384x512_CROP_30FPS] = radar_16384x512_crop_30fps,
    [RADAR_MODE_1024x640_CROP_30FPS] = radar_1024x640_crop_30fps,
    [RADAR_MODE_2048x640_CROP_30FPS] = radar_2048x640_crop_30fps,
    [RADAR_MODE_4096x640_CROP_30FPS] = radar_4096x640_crop_30fps,
    [RADAR_MODE_8192x640_CROP_30FPS] = radar_8192x640_crop_30fps,
    [RADAR_MODE_16384x640_CROP_30FPS] = radar_16384x640_crop_30fps,
    [RADAR_MODE_1024x1024_CROP_30FPS] = radar_1024x1024_crop_30fps,
    [RADAR_MODE_2048x1024_CROP_30FPS] = radar_2048x1024_crop_30fps,
    [RADAR_MODE_4096x1024_CROP_30FPS] = radar_4096x1024_crop_30fps,
    [RADAR_MODE_8192x1024_CROP_30FPS] = radar_8192x1024_crop_30fps,
    [RADAR_MODE_16384x1024_CROP_30FPS] = radar_16384x1024_crop_30fps,
};
```

7.2 Serializer/De-serializer configuration

The driver for GMSL (max929x.ko) is provided as source code in this package. The driver will be compiled and loaded automatically by *init_jetson.py*.

The kernel module contains the configuration of GMSL i.e. upon loading it detects and configures the GMSL boards as needed.

7.3 iRFE-Jetson

The iRFE-Jetson is a comprehensive software integration package for the CARKIT 88C0 radar system, connected to the Nvidia Jetson platform via GMSL2 (SerDes) technology, as shown in the [Hardware build-up](#)¹ section. The iRFE-Jetson provides control and interface capabilities for the CTRX8188 radar chip on the CARKIT 88C0 module, enabling users to write custom C applications with the necessary configuration and control of the CARKIT 88C0 module.

The iRFE-Jetson includes:

- iRFE-driver (core component)
 - Access to firmware commands
 - Convenience functions
- Platform Abstraction Layer for the Nvidia Jetson
 - Communication interfaces
 - SPI, I2C, GPIO
 - Timing services
 - Get current time, delay, and timeout
- Hardware-specific implementations for the CARKIT 88C0
 - GPIO pin definitions, SPI configuration, device tree integration

The C-code example application is located in the 'src\demo_app\boards\carkit_88c0\example' directory. The application shows the complete initialization, configuration, and operation of the CARKIT 88C0 module.

It includes:

- Complete system initialization (from power-on to operational state)
- Coordinated radar operations (with synchronized measurement cycles)
- Power management (with low-power modes between measurements)
- PMIC configuration (for proper voltage supply management)
- TX calibration procedures (for optimal radar performance)

To build and execute the example, execute the following steps:

```
# Enter the example directory
cd <root_dir>/src/demo_app/boards/carkit_88c0/example

# Build using make
make

# Execute the example
./bin/carkit88c0_example
```

The user can use the existing example application as a template to develop a custom application for a specific use case.

7.4 Strata

To control the Jetson from Matlab, at least Strata 3.1.x is needed. For details of the Strata interface, please refer to the SW user guide provided with the Strata package under <StrataBasePath\documentation>.

1. <https://confluencewikiprod.intra.infineon.com/spaces/ASRSSET/pages/2360607410/CARKIT+2C0+User+Guide#CARKIT2C0UserGuide-Hardwarebuild-up>

In a typical scenario, Jetson should be configured for IP address 169.254.1.101 and the Matlab host PC for IP address 169.254.1.1.

The Jetson side of the Matlab interface is provided as a Docker container. The Docker container must be build and can then be launched.

1. Once the initialization (see *Start System*) is successful, start the board simulator. Board simulator can be launched using the following steps:
 - I. Navigate to the *strata-board-simulation* folder in the Jetson
 - II. Execute: `docker load -i strata-board-simulator-carkit-88c0.tar.gz`
 - III. Execute: `docker run -i --rm -p 55055:55055 -p 55056:55056 --privileged board-simulator-carkit-88c0`
2. On the host pc:
 - I. When connected to the Jetson via SSH, ensure tunneling of local ports 55055 & 55056 via SSH to the Jetson.

Strata on Host side comes with an example `Standalone_example_88c0.m` (<strataBasePath\matlab\examples>) that can directly be used with the board simulator.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition yyyy-mm-dd

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2023 Infineon Technologies AG.

All Rights Reserved.

Do you have a question about this document?

Email: erratum@infineon.com

Document reference

AppNote Number

IMPORTANT NOTICE

The information contained in this application note is given as a hint for the implementation of the product only and shall in no event be regarded as a description or warranty of a certain functionality, condition or quality of the product. Before implementation of the product, the recipient of this application note must verify any function and other technical information given herein in the real application. Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind (including without limitation warranties of non-infringement of intellectual property rights of any third party) with respect to any and all information given in this application note.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.